

Studio DB reale (_{dwh}) + Piano di allineamento

Analisi del database **reale** (`dwh.sql`, dump custom PostgreSQL 17, schema `dwh`) e piano per allinearlo all'app **senza modificare React**, tramite viste di compatibilità in schema `public`.

1. Il modello reale è EAV (questionari), NON dimensionale

Il DB reale ha **22 tabelle** nello schema `dwh` ed è organizzato attorno ai **questionari**:

- `ft_questionari_globale` (+ varianti `ft_questionari`, `_2024`, `_tableau*`) = **tabella dei fatti EAV**: progetto, ente, cf_ente, faseelaborazione, raw_element, valore, valore_num, elementid, rilevazione → **una riga per ogni risposta/metrica** di un ente in una rilevazione.
- `lk_questionari_elements` (64) = dizionario degli element (`raw_element` → dimensione + descrizione).
- `lk_anagrafica_indicatori` (27) = indicatori con **query SQL di calcolo già pronta** (colonna `query`) + `dim_sigla` (D1–D6) + `ruolo_id`.
- `lk_questionari_kpi` = definizioni KPI.
- `lk_questionari_enti` = anagrafica enti reale (con `codice_ipa`, `codice_fiscale_ente`, `codice_istat`, `codice_comune_istat`, `tipologia`, `classe_di_popolazione`).
- `lk_comuni`, `lk_shp_*` = comuni e geografie.

Al contrario, il **Prototipo** usa ~150 tabelle **denormalizzate** (`dw_*`, `ca_*`, `ft_sipo_*`) che **non esistono** nel DB reale.

2. Conseguenza

Contro il DB reale, allo stato, **nessun grafico funziona** (mancano le tabelle `dw_*`). Ma **tutti i dati ci sono**, in forma EAV. Vanno **trasformati**.

3. Chiave ente = CODICE FISCALE

Nel fatto reale l'ente è identificato da `cf_ente` (codice fiscale). `lk_questionari_enti` ha `codice_fiscale_ente`, `codice_ipa`, `codice_istat`. → Conferma la scelta del cliente: **la chiave canonica è il CF**. L'`id_ente` atteso dall'app sarà un surrogato derivato da `lk_questionari_enti.id`, e le viste dei fatti mapperanno `cf_ente` → `id_ente`.

4. Gli indicatori hanno già la formula

`lk_anagrafica_indicatori.query` contiene, per ognuno dei 27 indicatori, la SELECT su `ft_questionari_globale` con filtri su `raw_element/elementid` e `cf_ente`, e placeholder: `#progetto#`, `#rilevazione#`, `#codicefiscale#`. Esempi: - D1 id=2 (% adozione modello): `SUM(case when elementid='1.1' then valore_num else 0 end)/count(distinct ente)` - D2 id=15 (assunti TI): `sum(valore_num) where raw_element like '2.1'` - D3 id=17 (concorsi banditi): `... raw_element like '3.1'`

→ Ricostruire gli indicatori = eseguire queste query sostituendo i placeholder (progetto/rilevazione/CF).

5. Architettura di allineamento

[`dwh.*` (EAV, dati reali)] -> [`public.dw_*` (VISTE/MV di compatibilità)] -> PostgREST/Supabase -> R

- Nuovo progetto Supabase (isolato dal progetto attuale)
- Restore del dump nello schema `dwh`

- Viste `public.dw_*` che pivotano/trasformano l'EAV nel contratto atteso dai grafici
- App puntata al nuovo progetto (solo variabili `.env`)

6. Piano a fasi

- **A. Ambiente:** nuovo progetto Supabase + `pg_restore` (v17) del dump nello schema `dwh`.
- **B. Mapping:** matrice `raw_element/elementid` → colonna `dw_*` (dai dizionari `lk_questionari_elements` e dalle query indicatori). Vedi `analisi_dwh/`.
- **C. Viste** (`public`):
 - C1 `dw_ente` (pronta — vedi `views/dw_ente.sql`)
 - C2 pilota indicatori/Executive da `lk_anagrafica_indicatori.query`
 - C3 estensione a tutte le `dw_*` usate dai grafici (vedi `MAPPATURA_TABELLE_GRAFICI` SC)
- **D. Switch & validazione** grafico per grafico; gap → `DemoDataBadge`.

7. Note operative

- Il dump è **custom PG17**: restore con `pg_restore v17` (`--no-owner --no-privileges`).
- Le viste in `public` vanno esposte via PostgREST con GRANT/RLS corretti per `anon/authenticated`.
- I fatti reali sono per **rilevazione** (es. 2024-12) e **progetto** (es. GRU, RiVa): le viste devono scegliere/parametrizzare rilevazione e progetto.

7-bis. VALIDAZIONE su dati reali (eseguita)

Dump ripristinato in un PostgreSQL 17 locale e viste costruite/testate sui **dati veri**:

Elemento	Esito
Restore dwh	OK (solo <code>lk_shp_*</code> geografiche saltate: richiedono PostGIS)
Dati	<code>lk_questionari_enti</code> 23.594 · <code>lk_comuni</code> 8.093 · <code>ft_questionari_globale</code> 36.992 · <code>indicatori</code> 25 · <code>elements</code> 62
<code>public.dw_ente</code>	■ costruita e validata (23.594 enti) → <code>views/dw_ente.sql</code>
<code>public.v_kpi_ente_wide</code>	■ pivot EAV→wide validato: 619 righe, 264 enti monitorati , 78 metriche → <code>views/v_kpi_ente_wide.sql</code>
Calcolo indicatore reale	■ es. assunti TI (<code>raw_element</code> 2.1, GRU, 2024-12) = 2359; Brescia 149, Padova 99...

Nota: solo **264 enti** hanno dati di questionario (su 23.594 in anagrafica) — è il set realmente monitorato. **Chiave di join dei fatti = `cf_ente` (codice fiscale)** → `id_ente` risolto via `lk_questionari_enti`.

Viste prodotte (in `views/`)

- `dw_ente.sql` — anagrafica ente (validata)
- `v_kpi_ente_wide.sql` — pivot base per-ente da cui derivare `dw_kpi_rilevazione` e i fatti (validata)

Prossimi passi

- Da `v_kpi_ente_wide` derivare `public.dw_kpi_rilevazione` (rinominare `q_<dim>_<n>` → nomi attesi) e le `dw_*` dei fatti.
- Ricostruire `dw_verifica_indicatori` dalle 25 query di `lk_anagrafica_indicatori` (per-ente).
- Esporre le viste via PostgREST (GRANT SELECT a `anon/authenticated`).

8. Allegati di analisi (in `analisi_dwh/`)

- `elements.txt` — dizionario dei 64 element
- `indicatori.txt` — i 27 indicatori con le query di calcolo

9. AGGIORNAMENTO — Viste fatti KPI (validate sui dati reali)

9.1 `dw_kpi_rilevazione.sql` (NUOVA — validata, 115 enti)

- Deriva da `v_kpi_ente_wide` filtrata sulla campagna più recente e completa, allineata al dizionario `lk_questionari_elements` (2025-04): `progetto = 'progetto_gru'`, `rilevazione = '2025-12'`.
- Mappa ~70 colonne `q*` del contratto app (`types.ts`) ai `raw_element` reali:
- Binari: da `valore_num` (1 → Sì; per `q1_1_adozione_modello`: 1 → Formalmente).
- Numerici: `valore_num` castato a `text` (il contratto app è `text`).
- Totali/denominatori (`q6_tep_personale`, `q6_totale_donne`, `q6_sw_hr_totali`, ...): derivati per somma dei sotto-elementi (es. `TI = 6.4.a+b+c+d`).
- Colonne senza sorgente reale (`q6_4_posti_vacanti_*`, `q6_12_donne_agile_pct`): `NULL + TODO`.

9.2 `dw_verifica_indicatori.sql` (NUOVA — validata, contratto)

- **Non interrogata da query live:** radar/indici Executive sono array demo statici in `executiveData.ts`; l'IAC è ricalcolato lato client da `dw_kpi_rilevazione` (`iacService.ts`).
- La vista mantiene il contratto: `anagrafica` (`id_ente`, `denominazione`, `tipologia`) e `organico_2023` popolati; indici compositi (`iac`, `iap`, `icec`, ...) esposti `NULL` — richiedono la **metodologia ufficiale di scoring** del committente.

9.3 Nota sulle campagne (schemi `raw_element` diversi)

- `GRU/2024-12`: schema vecchio (6.16 gg, 6.17.a-d, 6.18.a-d).
- `progetto_gru/2025-12`: allineato al dizionario 2025-04 (6.16.a/b, 6.17, 6.18, 6.19.a-d, 6.20.a-d).
- Le viste usano `progetto_gru/2025-12`. Rilevazione/progetto **parametrizzabili** nel CTE `w`.

9.4 Prossimi passi

- Costruire le `dw_*` occupazionali (`dw_occupazione`, `dw_assunti`, `dw_cessati`, `dw_eta`, `dw_formazione`, `dw_modalita_lavoro`) dal dump reale Conti Annuali `ca_52.sql` (schema `ca`).
- Esporre tutte le viste via PostgREST (`GRANT SELECT anon/authenticated`) + RLS su claim JWT.

10. AGGIORNAMENTO — Viste occupazionali da Conti Annuali (ca) — validate

Importato il dump reale `ca_52.sql` (schema `ca`, Conti Annuali, PostgreSQL 16→17, 54 tabelle) e costruite 6 viste materializzate, **validate sui dati reali** (anni 2012-2024):

Vista	Sorgente	Righe
<code>dw_occupazione.sql</code>	<code>ft_occupazione</code>	1.211.654
<code>dw_assunti.sql</code>	<code>ft_assunzioni</code>	221.737
<code>dw_cessati.sql</code>	<code>ft_cessazioni</code>	430.711
<code>dw_eta.sql</code>	<code>ft_eta</code>	2.836.412
<code>dw_formazione.sql</code>	<code>ft_formazione</code>	491.209
<code>dw_modalita_lavoro.sql</code>	<code>ft_modalita_lavoro_flessibile</code>	95.896

10.1 Chiave ente (istituzione) — mapping validato

La colonna app istituzione (int) = `dw_ente.id_ente`. Percorso di join (vedi `_ca_map_note.md`):

```
ca.<fact>.ISTITUZIONE = ca.lk_istituzioni.istituzione_id
ca.lk_istituzioni.CODI_FISCALE = dwh.lk_questionari_enti.codice_fiscale_ente
dwh.lk_questionari_enti.id = dw_ente.id_ente (= istituzione)
```

Overlap verificato: **12.274** codici fiscali CA combaciano con l'anagrafica enti dwh.

10.2 Note di mappatura

- Nomi colonna reali in MAIUSCOLO/virgolette (es. "PERSONALE_TEMPO_PIENO_DONNE" → tp_donne).
- dw_occupazione.macrocat: non presente in ft_occupazione → NULL.
- dw_formazione.ore_media_*: la fonte CA esprime la media in **giornate** (FORM_MEDIA_*), mappata su ore_media_d/u per rispettare il contratto app.
- categoria/contratto/qualifica/causale: esposti come **codici** (join descrizioni via lookup dw_causali/dw_comparto_contratto da costruire in seguito).

10.3 Spot-check (Comune di Trieste, id_ente 8843, 2023)

- Occupazione: 2.158 tempo pieno (D 1.453 / U 705) + 249 part-time.
- Assunti: 237 (D 123 / U 114) su 13 righe contratto/qualifica.

10.4 Ambiente di test (pod effimeri)

Lo script `docs/setup_local_pg.sh` reinstalla PostgreSQL 17 (PGDG), scarica i dump, ripristina `dwh` + `ca` e ricrea tutte le viste `dw_*` in ordine di dipendenza. Da rieseguire se il pod viene resettato.

11. AGGIORNAMENTO — Viste lookup descrizioni (validate)

Costruite 3 viste di lookup per tradurre i codici in descrizioni leggibili:

Vista	Sorgente	Righe	Uso live
dw_causali.sql	vw_lk_causali_assunzione + vw_lk_causali_cessazione	18	Sì (assunti/cessati)
dw_comparto_contratto.sql	lk_mappa_comparti_contratto	49	No (contratto)
dw_fascia_eta.sql	derivata dai 12 codici reali di ft_eta	12	Sì (eta)

11.1 dw_causali — chiave di join corretta

I fatti usano codici **prefissati** (A23 assunzione, C01 cessazione), NON i codici nudi di `lk_causali` (23, 01). La fonte corretta sono le viste `ca.vw_lk_causali_*` che espongono `causale_id` = codice prefissato identico ai fatti (il prefisso A/C risolve anche l'ambiguità del codice 28). Coverage join validata: **9/11** codici (es. "NOMINA DA CONCORSO" 165.054 assunti 2023). I codici A26, A40 (+2 cessazione) non hanno descrizione in nessuna tabella del dump → fallback al codice (comportamento già previsto dai service).

11.2 dw_fascia_eta

Join validato **12/12** con `dw_eta.fascia_eta` (E0..E68); etichette e ordinamento (`eta_min`) generati per le sezioni demografiche.

11.3 Descrizioni categoria/contratto/qualifica

`dw_comparto_contratto` fornisce `comparto+contratto`. Le descrizioni di CATEGORIA/QUALIFICA sono disponibili in `ca.lk_comparti_categorie_contratti` / `ca.lk_contratti_categorie`: potranno alimentare eventuali lookup aggiuntivi se i grafici le richiederanno.

12. AGGIORNAMENTO — Qualifiche estese + Passaggi qualifica (validate)

Vista	Sorgente	Righe	Uso live
dw_qualifiche.sql	lk_comparti_categorie_contratti	3.110	No (lookup ruoli)
dw_passaggi_qualifica.sql	ft_passaggi_qualifica	199.608	Sì (progressioni)

12.1 dw_qualifiche — ruoli per esteso

Chiave di join validata **100%** (2019/2019 ruoli occupazione):

```
dw_occupazione.qualifica = dw_qualifiche.categoria      (= CODI_QUALIFICA)
dw_occupazione.contratto = dw_qualifiche.cod_contratto
```

La colonna categoria contiene il **codice qualifica** (il contratto app non ha una colonna esplicita per esso), descrizione = ruolo esteso (es. "POSIZIONE ECONOMICA C2"), **macrocategoria** = raggruppamento (DIRIGENTI, ecc.).

12.2 dw_passaggi_qualifica — progressioni

Mapping diretto da ca.ft_passaggi_qualifica. Il TIPO_PASSAGGIO reale è V/O; tradotto nell'adapter in Verticale/Orizzontale per compatibilità col frontend (/vert/i). Volumi validati (2024: 45.355 verticali, 164.737 orizzontali).

12.3 ■■ InPA e Syllabus — SORGENTE DATI ASSENTE

Le sezioni **InPA** (dw_inpa_bandi, dw_inpa_candidati, dw_lp_graduatorie, dw_ptfp_reclutamento) e **Syllabus** (dw_syllabus_pa, dw_syllabus_catalogo, dw_syllabus_partecipazioni) **non hanno alcuna tabella sorgente** nei due dump forniti (dwh.sql, ca_52.sql) — verificato su tutti gli schemi. Sono sistemi esterni (portale reclutamento InPA e piattaforma competenze Syllabus). **Per popolarle con dati reali serve che il committente fornisca i relativi dump/estrazioni.** Nel frattempo restano alimentate dai dati demo/JSON già presenti nell'app.